Web brocks v2

02 May 2025 16:27

Here is a professional and guideline-oriented version of your coding guidelines based on the image, reformatted as bullet points and written in a consistent tone:

Web Bricks Coding Guidelines

To maintain code quality, consistency, and reliability across the project, adhere to the following best practices:

- **Use specific types in TypeScript**
  - Avoid using any. Always define the expected data type explicitly (e.g., use let data: string = "value").
  - This improves type safety and prevents runtime errors.
- **Ensure unit and integration tests are written**
  - Do not skip writing tests. Include both unit and integration tests for all key features to ensure system reliability and regression coverage.
- **Manage fixture data correctly in tests**
  - Avoid changing fixture data directly in the test file. Instead, manage or update fixture values within the dedicated fixture file to improve maintainability and separation of concerns.
- **Test for meaningful values rather than checking for null or undefined**
  - Avoid assertions like not.toContain('undefined'). Focus on testing against expected values using meaningful data.
    Example:
    expect(minichartLink?.getAttribute('href')).toEqual('some value');
- **Prefer nullish coalescing (??) or optional chaining (?.) over logical operators**
  - Use ?? to provide default values when dealing with null or undefined.
  - Use || only if you need to fall back on any falsy value (false, 0, '', null, undefined).
  - Example:

```
function foo(bar) {
  bar = bar ?? 55;
  console.log(bar);
}

foo(22); // 22
foo(null); // 55
```

- **Batch state updates using a single update function**
  - When updating multiple pieces of state, avoid calling separate update functions that trigger multiple DOM updates.
  - Use a unified function like updateStateMulti() to reduce unnecessary re-renders.

```
this.updateStateMulti({
  isWatchlistButtonClicked: true,
  isDropdownOpen: false
});
```

- **Avoid hardcoded strings in test assertions**
  - Use fixture data to assert expected values instead of hardcoded strings to prevent brittle tests.
  - Example (bad):
    expect(component.getState().lastTransactionDate).toBe('01.03.2025');
- **Mock API calls with a complete set of test values**
  - Do not ignore mocking for all possible API scenarios. Ensure all expected inputs and edge cases are covered.
  - This avoids unexpected warnings or test failures such as: WARN LOG: 'Unmatched GET to /web...'

Here is the second part of your **Web Bricks Coding Guidelines**, written in the same professional

tone and structured style as the first part you approved earlier:

Web Bricks Coding Guidelines (Continued)

To maintain code clarity, consistency, and testability across the project, adhere to the following additional practices:

- Always import modules using the complete path
  - Avoid partial or relative paths when shared modules or styles are involved.
  - Example:
    import { DISABLE_CLASS, HIDDEN_CLASS } from 'wb-business/portfolio-shared/common/css';
- Reuse expected response objects in test cases using fixtures
  - Move such objects to the fixture files and reference them in your tests to improve maintainability.
- Replace magic numbers with clearly named constants
  - Define and use constants to represent values like page size, page number, or sort order instead of hardcoding them.
- Extract repeated logic into utility methods
  - When the same logic (like validation or conditional checks) is used across multiple places, encapsulate it in a utility function for reusability.
- Remove async from tests that do not use await
  - Avoid unnecessary async usage in test declarations if no asynchronous operation is involved.
- Avoid using any when working with HTML elements
  - Use precise type assertions (e.g., HTMLInputElement) to ensure type safety.
    Example:
    const vpNameInput =
    document.getElementById('newVirtualPortfolioNameModalInput') as HTMLInputElement;
- Avoid redundant method names like dispatchEventByName
  - Use clearly named functions and leverage parameter values directly when dispatching events.
    const dispatchWatchlistEvent = (eventName: string) =>
    window.dispatchEvent(new CustomEvent(eventName, { bubbles: true }));
- Use template literals for multi-line strings
  - Avoid string concatenation using + across lines. Template literals offer cleaner syntax and better readability.